

Configuration Drift Detector

Catching silent code ↔ environment mismatches before they cause an outage

Project Report

Team Members
Boomika A
Abarna A
Akashiya A

Domain: DevOps Tooling · Configuration Management · Static Analysis

Date: September 2026

1. Problem Statement

Configuration drift between code and its deployment environments is one of the most common, and most avoidable, causes of production outages. A developer adds a new environment variable locally, wires it into the code, tests it, and merges the change. It works — because their local `.env` file has the key. Days later a staging or production deploy crashes on boot with a cryptic error, because nobody updated that environment's configuration to match.

No single person decides this should happen. It accumulates through small, individually reasonable changes: a renamed variable, a value that is optional in development but required in production, a secret rotated in one place and forgotten in another. Detecting this today is manual — someone greps the codebase for `process.env` references, someone else diffs `.env` files by eye, and with luck the mismatch is caught before a deploy surfaces it.

Goal: build a tool that treats configuration as a contract between code and environment, and checks that contract automatically — scanning the codebase for every configuration key it actually references, scanning each environment's configuration source for what is actually defined, and reporting the drift between the two as a structured report anchored to real `file:line` references, without ever printing a secret value.

1.1 Why this matters

- Configuration bugs are invisible in code review and in local testing — they only surface once a specific environment is booted.
- They tend to surface at the worst time: after merge, during a deploy, often outside working hours.
- The fix is usually a one-line environment update, but the cost is an incident, a rollback, and lost trust in the deploy pipeline.
- This class of bug is systemic, not a one-off mistake — which means it needs a systemic (automated, repeatable) check, not a checklist.

2. Objectives

- **Discover** every configuration key the codebase actually reads, with the exact file and line it is read from.
- **Discover** every configuration key actually defined for each environment (dev, staging, production), from whatever source each environment uses.
- **Diff** the two sets per environment and classify every mismatch into a clear category — never leave a finding unclassified.
- **Never print secret values.** The tool reasons about key names and shapes, never the underlying secret content.
- **Produce a structured, actionable report** — grouped by severity and environment, anchored to `file:line`, suitable for a PR check or CI gate — not a wall of text.

3. Categories of Drift Detected

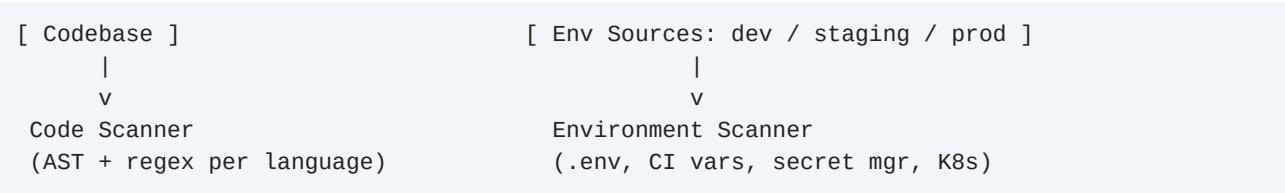
Category	Definition	Example	Severity
Missing Key	Code reads a key that has no corresponding entry in the environment's configuration source.	PAYMENTS_GIVEBOOK_SECRET is present in staging.env but not in prod.env.	Low
Orphaned Key	A key is defined in an environment's configuration source but is never referenced in the codebase.	UNUSED_SECRET is defined in prod.env but not used anywhere.	Low
Format Mismatch	The key exists on both sides, but its declaration or expected type differs.	MAX_RETRIES is declared as integer in staging and as string in prod.	Medium
Cross-Environment Inconsistency	A key is defined in one environment but not another, or has different values.	FEATURE_FLAG is set to true in staging and false in prod.	High
Renamed / Near-Duplicate Key	A key in code closely resembles an existing key, suggesting a rename or typo.	DB_CONNECTION_URL is in code, while DB_URL is in staging.env.	Medium

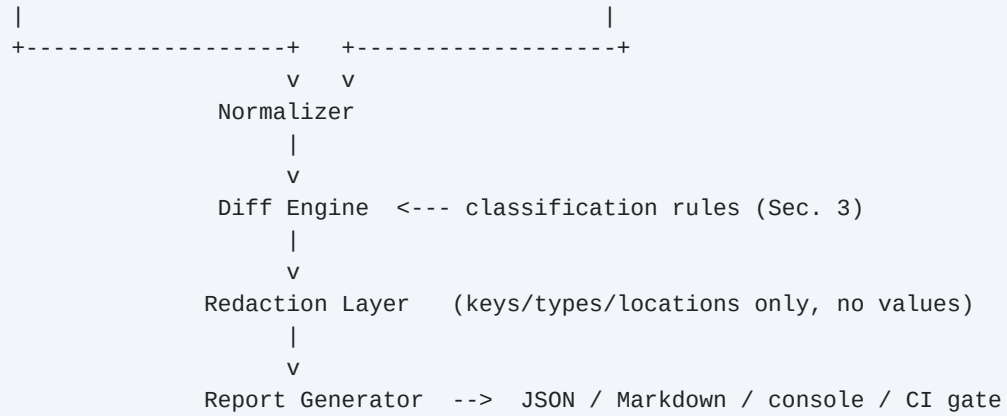
4. System Architecture

The tool is organized as a pipeline of independent, composable stages so that new languages, frameworks, or configuration sources can be added without touching the diff or reporting logic.

- 1. Code Scanner** — Walks the repository and extracts every configuration-key reference, with its exact file path and line number, plus inferred usage context (required/optional, default value, expected type).
- 2. Environment Scanner** — Reads each environment's configuration source(s) — .env files, CI/CD variable groups, secret managers, Kubernetes ConfigMaps/Secrets, cloud parameter stores — and extracts the set of key names actually defined, never their values.
- 3. Normalizer** — Reconciles naming conventions across sources (case, prefixes, nesting) so the same logical key is recognized whether it appears as MAX_RETRIES, max_retries, or app.max_retries.
- 4. Diff Engine** — Computes, per environment, the set difference and shape comparison between 'keys code expects' and 'keys the environment defines', and classifies each finding into one of the drift categories in Section 3.
- 5. Redaction Layer** — Sits between every scanner and the report — guarantees that only key names, types, and locations ever reach the output, never values. This is enforced structurally, not left to convention.
- 6. Report Generator** — Renders the classified findings into a structured report (JSON for machines, Markdown/HTML/console for humans), grouped by environment and severity, each finding anchored to a file:line reference.

4.1 Pipeline diagram (textual)





5. Code Scanner — Extracting Referenced Keys

The code scanner must find configuration reads without false positives, and record enough context to know whether a missing key is actually a problem (e.g. a key with a hardcoded default is not critical if missing).

5.1 Detection patterns per ecosystem

Language / Framework	Patterns matched
Node.js / JavaScript / TypeScript	<code>process.env.KEY</code> , <code>process.env['KEY']</code> , destructured <code>const { KEY } = process.env</code>
Python	<code>os.environ['KEY']</code> , <code>os.environ.get('KEY', default)</code> , <code>os.getenv('KEY')</code> , <code>pydantic / dotenv Settings classes</code>
Go	<code>os.Getenv("KEY")</code> , <code>os.LookupEnv("KEY")</code> , <code>viper.GetString("key")</code>
Java / Spring	<code>@Value("\${key}")</code> , <code>System.getenv("KEY")</code> , <code>application.yml placeholders</code>
Ruby	<code>ENV['KEY']</code> , <code>ENV.fetch('KEY', default)</code>
Docker / Compose / K8s manifests	<code>environment:</code> , <code>envFrom:</code> , <code>valueFrom.secretKeyRef.name</code> / <code>configMapKeyRef.name</code>
IaC (Terraform)	<code>var.key</code> references and <code>environment</code> blocks in provider resources

5.2 Why AST parsing over plain grep

A pure regex grep for `process.env` over-reports: it matches comments, string literals, and disabled code, and it cannot see whether a call site provides a default value or is inside a try/catch that already handles the missing-key case. The scanner therefore uses a language-aware AST parser per ecosystem (e.g. a JS/TS parser for Node code, Python's `ast` module) to resolve the call site precisely, and falls back to a scoped regex pass only for file types without a practical parser (YAML/Helm templates, Dockerfiles).

5.3 Captured metadata per reference

- **Key name** — normalized form of the configuration key.
- **File and line** — exact source location, e.g. `src/payments/webhook.js:42`.
- **Required / optional** — inferred from whether a default value or fallback branch is present.
- **Expected type** — inferred from usage: `parseInt/Number()` nearby implies numeric, `JSON.parse` implies structured, boolean comparisons imply boolean.
- **First-seen commit / PR** (optional, if git history is available) — helps attribute a newly introduced key to the change that added it.

6. Environment Scanner — Extracting Defined Keys

Each environment can source its configuration from more than one place. The scanner treats each environment as a named collection of configuration sources, and merges the key sets according to that environment's known precedence order (e.g. secret manager overrides ConfigMap overrides .env defaults).

Source type	What is extracted	Value handling
.env / .env.staging / .env.production files	Key names on the left-hand side of each assignment	Secrets are never read into memory beyond a hash for change-detection
CI/CD variable groups (GitHub Actions secrets, GitLab CI variables)	Variable names (as variables)	Under API/CLI listing API calls request metadata only, never a value-reveal endpoint
Cloud secret managers (AWS Secrets Manager, GCP Secret Manager, Vault)	Secret IDs / Secret paths	List-only API calls; get-value calls are never made
Kubernetes ConfigMaps & Secrets	Keys under .data / .stringData	Secret values are base64 blobs the tool never decodes
Cloud parameter stores (AWS SSM Parameter Store)	Parameter names under the environment's namespace	With encryption is never set to true

6.1 Secret safety guarantee

The environment scanner is built so that it is structurally incapable of reading a secret's value into the report path: every provider integration uses a list/describe API call that returns names and metadata only (e.g. AWS secretsmanager:ListSecrets rather than GetSecretValue). Where a source mixes names and values in one line (a raw .env file), the parser discards the right-hand side immediately after computing an optional non-reversible hash used only to flag when a value has silently changed, never to display it.

7. Diff Engine — Rules and Precedence

For each environment, the diff engine computes three sets: **Used** (keys referenced in code), **Defined** (keys present in that environment's merged configuration), and compares them.

- **Missing** = Used – Defined, filtered to only *required* keys (optional keys with a safe default are downgraded to Info, not Critical).
- **Orphaned** = Defined – Used, excluding keys explicitly allow-listed as intentionally unused (e.g. reserved for a feature flag rollout).
- **Format Mismatch** = keys present in both sets whose inferred type/shape from code disagrees with the value's detected shape in the environment source.
- **Cross-Environment Inconsistency** = a key present in ≥ 1 environment's Defined set but absent from another, cross-checked against an optional per-environment exclusion manifest.

7.1 Severity assignment

Finding	Default severity	Rationale
Missing + required + no default	Critical	Will crash the service on boot or first use.
Missing + required + has fallback path	Warning	Degraded behavior likely, not a hard crash.
Orphaned key	Info	No functional risk, but a source of confusion and stale secrets.
Format mismatch	Warning	May pass validation but produce wrong runtime behavior.
Cross-environment inconsistency	Info / Warning	Depends on whether the asymmetry is documented as intentional.

8. Sample Drift Report Output

The report is anchored to file:line references and grouped by environment and severity. A machine-readable JSON form drives CI gating; a Markdown/console form is for humans.

```
{
  "environment": "staging",
  "scanned_at": "2026-09-10T04:12:00Z",
  "summary": { "critical": 1, "warning": 2, "info": 1 },
  "findings": [
    {
      "category": "MISSING_KEY",
      "severity": "critical",
      "key": "PAYMENTS_WEBHOOK_SECRET",
      "referenced_at": "src/payments/webhook.js:42",
      "required": true,
      "has_default": false,
      "message": "Key is read at webhook.js:42 with no default; not defined in staging."
    },
  ],
}
```

```
{
  "category": "FORMAT_MISMATCH",
  "severity": "warning",
  "key": "MAX_RETRIES",
  "referenced_at": "src/http/client.py:17",
  "expected_type": "integer",
  "message": "Code parses this as an integer; staging value is non-numeric."
},
{
  "category": "ORPHANED_KEY",
  "severity": "info",
  "key": "LEGACY_SMTP_HOST",
  "defined_in": "staging.env:9",
  "message": "No references found in codebase; last used code path removed."
}
]
```

8.1 Console / PR-comment rendering

CONFIG DRIFT REPORT — staging

=====

[CRITICAL] Missing key: PAYMENTS_WEBHOOK_SECRET
used at src/payments/webhook.js:42 (required, no default)
status NOT DEFINED in staging

[WARNING] Format mismatch: MAX_RETRIES
used at src/http/client.py:17 (expected: integer)
status defined in staging, value is non-numeric

[INFO] Orphaned key: LEGACY_SMTP_HOST
defined staging.env:9
status no code reference found

Summary: 1 critical, 1 warning, 1 info -> exit code 1 (blocks deploy)

9. Where It Fits — Workflow Integration

- **Pre-merge (PR check):** runs on every pull request; fails the check if the PR introduces a key with no corresponding entry in the target deploy environment(s), forcing the config update into the same review as the code change.
- **Pre-deploy gate:** runs immediately before a staging or production deploy as a required pipeline step; a Critical finding blocks the deploy rather than letting the service crash on boot.
- **Scheduled audit:** runs on a daily/weekly schedule against long-lived environments to surface orphaned keys and slow-accumulating drift that no single PR would have triggered.
- **Local pre-commit (optional):** a lightweight subset (missing required keys only) can run locally so a developer sees the gap before ever opening a PR.

10. Proposed Technology Stack

Component	Technology
Code scanning (JS/TS)	Babel parser / TypeScript compiler API (AST traversal)
Code scanning (Python)	Built-in ast module
Code scanning (Go, Java, Ruby, YAML/Docker)	Tree-sitter grammars for a unified multi-language AST layer
Environment source adapters	Provider SDKs: boto3 (AWS), google-cloud-secret-manager, hvac (Vault), kubernetes o
Diff engine / core logic	Python, with a pluggable rule-registry so new drift categories can be added
Report output	JSON (machine), Markdown/console (human), optional HTML for dashboards
CI integration	GitHub Actions / GitLab CI job, exit-code driven gating

11. Security & Safety Design Principles

- **Names only, never values:** every scanner interface is designed around list/describe operations; there is no code path through which a secret value can reach the report.
- **Least-privilege credentials:** the tool requests read-only, list-level IAM/RBAC permissions from each provider — it should not even be able to fetch a value.
- **Redaction at the boundary, not by convention:** the Redaction Layer (Section 4) is a structural checkpoint every finding passes through before serialization, not a rule a scanner author has to remember.
- **Audit logging:** every scan run is logged (what was scanned, not what was found in terms of values) for traceability.

12. Future Enhancements

- Auto-suggested fixes: generate a PR diff that adds a missing key with a safe placeholder value for a human to fill in.
- Historical drift trend dashboard — track how many findings accumulate per week per environment.
- Confidence-scored key-rename detection using string similarity plus usage-pattern matching.
- IDE plugin that flags a newly added configuration read in real time, before the PR is even opened.

13. Conclusion

Configuration is a contract between code and the environment it runs in, but today that contract is unwritten and unchecked — which is exactly why it breaks silently and expensively. By treating configuration keys as first-class, typed, located facts — extracted from code via AST analysis and from environments via read-only provider APIs — this tool turns an ad-hoc, manual, error-prone check into an automated, structured, CI-enforceable gate. The result is that a missing `PAYMENTS_WEBHOOK_SECRET` is caught at PR time, anchored to the exact line that needs it, instead of at 2am during a production boot failure.

Prepared by: Boomika A · Abarna A · Akashiya A